Portable Spec Kit — Spec-Persistent Development

Spec-Persistent Development for AI-Assisted Engineering

A single portable file that defines your complete development methodology — personalized from the first session. Drop it into any project and the AI agent detects your GitHub profile, adapts to your expertise and working style, follows your engineering standards, manages tasks, writes tests, and maintains context across sessions.

Author: Dr. Aqib Mumtaz, Ph.D.

Specialization: Ph.D. Computer Science — Artificial Intelligence

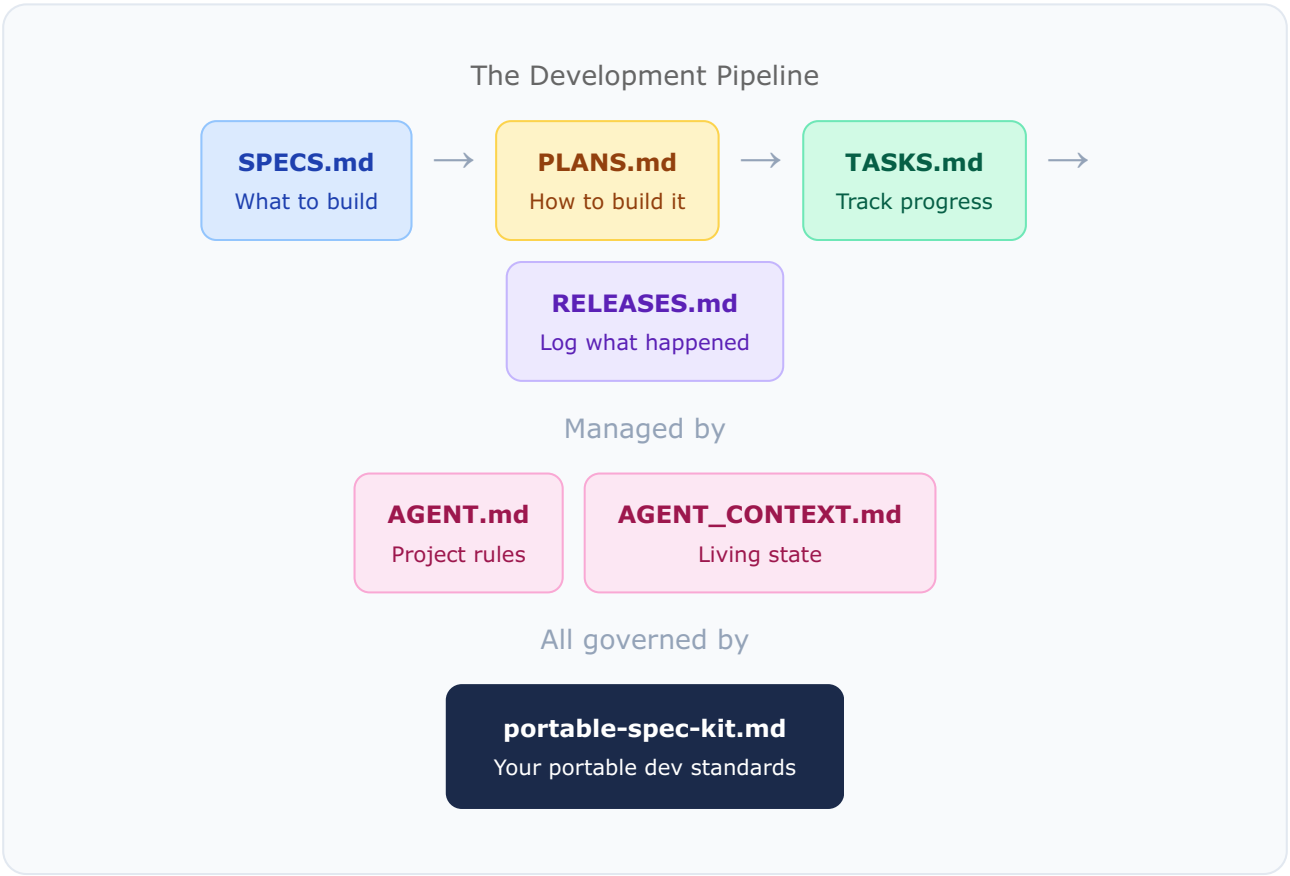
Research: Multimodal AI, Healthcare AI, Autonomous Surveillance

Version: v0.6.38

Date: May 2026

The Framework

Spec-Persistent Development — specs always exist and stay current, but never block. The framework doesn't enforce a methodology — it supports waterfall, agile, or mixed. The AI agent maintains living specifications throughout development, adapting to how you work.



11 Core Strengths

Strength	What It Means
🪶 **Lightweight**	Single markdown file. Zero dependencies. Zero install.
📦 **Portable**	One file → any repo. Works instantly. Symlinks for all agents.
👤 **Personalized**	GitHub profile auto-detect. Adapts to your expertise, communication style, and working pattern. Tailored AI behavior.
📋 **Spec-Persistent**	SPECS → PLANS → TASKS → RELEASES. Specs persist alongside your code. Your workflow, your choice.

🤖 **Agent-Agnostic**	Claude, Copilot, Cursor, Windsurf, Cline — one source, all sync.
🧠 **Context-Persistent**	AGENT_CONTEXT.md carries state across sessions. Pick up after weeks. Never lose context.
🔒 **Secure**	Never expose API keys — even if asked. Strict .env rules. Code security enforced.
🔄 **Non-Blocking**	Code first, specs later. Agent tracks silently. Never blocks the developer.
🏗️ **Self-Scaffolding**	Auto-creates 9 agent files, README, workspace context. 8 stack templates built-in. Ready to code in seconds.
✅ **Self-Validating**	Agent tests everything. Fixes before presenting. 98%+ coverage target.
🛡️ **Reliably Enforced**	3-layer enforcement: bash sync-check (18 deterministic checks, PSK001–PSK017), sub-agent critic (verbatim-quote gate), git + Claude Code hooks (block bad commits). Agent cannot skip quality gates. Works across all AI agents.

Reliability Architecture (v0.5.8+)

The kit guarantees consistency through three independent enforcement layers:

- **Bash critic:** `psk-sync-check.sh` runs 18 deterministic checks PSK001–PSK017 (version consistency across 10+ files, test counts, feature counts, SPECS staleness, R→F→T gate, script permissions, required directories, CHANGELOG/RELEASES content, ARD content, AGENT.md stack, README Latest Release + install list + agent table + flow table, flow doc content, critic prompts meta-check, secrets scanning). Mode auto-detection adapts checks to project type (kit/node/python/go/rust/generic). Runs in under 500ms. Silent on clean. Emergency bypass via `PSK_SYNC_CHECK_DISABLED=1` env variable.
- **Sub-agent critic:** `psk-critic-spawn.sh` uses a file-based protocol to spawn fresh-context sub-agents that verify semantic correctness — flow docs describe current state, release notes match actual changes. Five-iteration cap with human escalation.
- **Hooks:** `.claude/settings.json` PostToolUse warns on every edit if drift detected. `.git/hooks/pre-commit` blocks commits that fail sync-check. These run outside agent control — the agent physically cannot bypass them during commits.
  Install: `curl -fsSL https://raw.githubusercontent.com/aqibmumtaz/portable-spec-kit/main/install.sh | bash` — delivers the complete reliability infrastructure to your

project. Uninstall: `bash agent/scripts/psk-uninstall.sh` (preserves your pipeline files).

v0.5.23 — Full-Surface Documentation Coverage Analyzer

`agent/scripts/psk-doc-sync.sh` is the release-time tool that catches documentation drift *structurally* rather than by agent inspection. It extracts every feature from the current-minor CHANGELOG entry and checks its presence across five documentation surfaces: `agent/*.md` (architecture), `docs/work-flows/*.md` (user-facing workflows), `docs/research/*.md` (methodology paper), `ard/*.html` (architecture reference), and `README.md` (public overview). Each feature is classified COVERED (≥2 surfaces), PARTIAL (1 surface), or MISSING (0 surfaces) with a surface-legend code `[AFPDR]` and a heuristic-suggested target doc per gap.

The analyzer is mandatory at release Step 4 (Update Docs). The `STEP_4_FLOW_DOCS` and `STEP_9_VALIDATION` critic prompts now include an "AUTOMATED COVERAGE TOOL" clause that makes the sub-agent critic run the analyzer and treat every MISSING entry as STALE. Run with `--strict` for CI-style behavior (exit 1 on any gap). This addresses the structural reason v0.5.13–v0.5.18 shipped with flow-doc drift: critic prompts were only catching contradictions, never omissions.

v0.6.0 — AVACR: Reflex Evolves to Adversarial Goals + Sandbox Isolation + Peer-Exchange

F70 Reflex evolves from cooperative VACR (Verbal Actor-Critic Refinement) to **AVACR — Adversarial Verbal Actor-Critic Refinement Loop**. The loop gains explicit asymmetric goals, physical sandbox isolation, and machine-parseable peer-to-peer YAML exchange so humans read only the final signoff, never agent working output.

- **Adversarial goals with collaborative convergence:** QA-Agent's explicit goal is to FAIL the release (hunt exhaustively across 14 dimensions; default DENIED). Dev-Agent's goal is to FOOLPROOF the release against QA's hunt (fix findings + audit + harden sibling-class weaknesses). Convergence is QA hunting hard and finding zero blockers — not "Dev says it's done."
- **Physical sandbox worktree isolation:** QA operates in a git worktree at `reflex/sandbox/qa-pass-NNN/` with `agent/AGENT.md` and `agent/AGENT_CONTEXT.md` physically removed. Dev's project rules + living session narrative cannot bias QA's fresh-context investigation. Dev retains full-repo access. Dual-rooted access: `REFLEX_QA_WORKTREE` for reads, `REFLEX_QA_PROJECT` for invocations.
- **Peer-to-peer YAML exchange:** QA writes `findings.yaml` (priority + dimension + blast_radius + confidence + reproducibility + escalation + citable_quote + regression_vector + standard + recommendation per finding). Dev returns `dev-`

`result.yaml` (fixed / rebuttals / routed_to_human / escalated / deferred). Human reads only `signoff.md` (GRANTED or DENIED with exit criteria).

- **14-dimension adversarial hunt:** functional / edge-case / cross-pipeline / workflow / research-backed-quality / superficiality / production-readiness / security (OWASP) / performance / architectural-compliance / documentation / test-quality / tech-debt / readiness-affirmation.
- **4-persona testing:** every feature exercised as new-user / power-user / malicious-user / accessibility-user — different personas surface different failure classes.
- **Research capability + citable-quote honesty gate:** QA may invoke WebSearch/WebFetch for RFCs, OWASP Top 10, CVEs, benchmarks, academic papers. Every finding requires a citable quote with file:line reference + verbatim text that bash grep can verify. No hallucinated standards.
- **Dev peer-fixer protocol:** 4-bucket diagnosis (A/B/C/D with Bucket-D requiring counter-citation rebuttal); human-arbitration routing for cross-cutting / ADL-amendment findings (Dev does not auto-fix those); flaky-vs-deterministic distinction; sibling-class hardening mandated (fix the class, not just the instance).
- **Coverage declaration + regression-vector re-execution:** QA writes `coverage.md` listing tested × dimensions AND untested surface with reasons. Regression vectors preserved per-finding and re-executed verbatim each pass; failure upgrades priority one level (catches surface-patch vs root-cause fixes).
- **CHANGELOG truth-check mandate:** every bullet in current version's CHANGELOG must have a test case and observable evidence; unverifiable claims block signoff.
- **Consistency sweep:** `cycle` → `pass` (~30 refs), `Refine` → `Reflex` (3 stragglers with real install/update bugs surfaced and fixed).
- **Paper target:** *"Adversarial Verbal Actor-Critic Refinement (AVACR): Multi-Agent Verbal Reinforcement Learning with Asymmetric Goals for Spec-Grounded Post-Release Program Repair"* (ICSE / FSE / ASE). Note: "Adversarial" refers to inference-time asymmetric goals / red-teaming, not adversarial training (no gradient updates).

v0.6.1 — Kit self-evolution loop + MINIMAL template tier + 14 framework-gap closures

Dual-track release closing the empirical-kit-evolution loop end-to-end and adding a third response-template tier.

- **N33 adopter-side auto-submit:** consent-gated dispatcher opens anonymized kit issues at pass-end. Three modes (manual / prompt / auto) with four guards (consent, pass filter, 24h rate limit, anonymizer).
- **N38 maintainer-side intake automation:** weekly GitHub Action parses `avacr-eval` issues, drafts `agent/tasks/Gxx-*.md` kit tasks.
- **N44/N45 general-purpose runner:** mode-less `reflex/run.sh [--target <path>]`; `--kit-loop` chains prep-release + self-test with max 3 iterations.

- **Per-pass canonical trace (Q1) + trace continuity:** structured `findings.yaml` + verdict `signoff.md` per pass; next QA reads prior `findings.yaml` + `signoff.md` + cross-pass register at Phase 1.
- **MINIMAL response-format tier + Writing Style sub-section + breadcrumb rules 6a/6c/6d + pre-send self-check.**
- **Session-stack 3-tier scalability + promotion rule.**
- **14 framework-gap closures (G1-G15 less G11).**

v0.6.2 — Dev-branch isolation + protected-files write-ban + autoloop + history retention

Closes the reflex execution-model loop. QA was sandboxed; Dev now runs on an isolated branch with a 3-layer write-ban on the two spec-persistent-owned files. Autoloop unifies the iterate-until-GRANTED command for kit + user projects.

- **Dev-branch isolation:** every reflex pass runs Dev on `reflex/dev-cycle-NN-pass-NNN`. Fast-forward merge on GRANTED + auto-delete; unhappy branches retained (last 3) for review.
- **Protected-files write-ban (3-layer):** `agent/AGENT.md` and `agent/AGENT_CONTEXT.md` unmodifiable by reflex — prompt constraint + `gates.sh` diff check + `psk-sync-check.sh` pre-commit hook on reflex dev branches.
- **Sandbox purge after QA:** `file-bugs.sh` removes the current pass's sandbox immediately after findings extraction — Dev physically cannot read QA's workspace.
- **Autoloop generalization:** `bash reflex/run.sh` (default = autoloop) chains prep-release + pass + iterate until GRANTED. Single CLI surface; wrapper scripts retired.
- **Unified cycle-NN-pass-NNN naming:** every pass gets a cycle id (ad-hoc runs auto-assign one); single directory scheme.
- **History retention + reset:** `history_retention` config bounds disk (last 10 pass dirs + 3 dev branches + 3 sandboxes; register + summary.csv forever). `bash reflex/run.sh --reset [--confirm]` for full wipe.
- **Concurrency + empty-pass + intake parser update.**

v0.6.3 — Autonomous reflex run validation

Mechanical patch minted from an autonomous end-to-end reflex run on the v0.6.2 consolidation. Prep-release validated the full reflex architecture through the kit's own prepare-release pipeline — Dev-branch isolation, protected-files write-ban, autoloop, history retention, reset command, unified cycle naming all verified green. No feature changes.

v0.6.4 — Reflex convergence stopping + loop-resume fix + gates cache fix + token tracking

- **Convergence-based stopping (N56)** replaces "max 3 iterations" with real signals: GRANTED, REGRESSION, findings_floor, plateau (`patience_passes`), fix-rate drop (`min_fix_rate`), safety cap.
- `--loop --resume` **collision fix:** deferred flag resolution in `run.sh` ; `--loop --resume` now routes correctly to `loop-resume` mode instead of hijacking to `resume-qa` .
- `gates.sh` **cache-order bug:** rft-cache cleared at gate-runner entry so sequential gates don't see stale data.
- **Token tracking (** `reflex/lib/track-tokens.sh` **):** per-pass tokens aggregated into `reflex/history/token-usage.csv` with `tokens_per_finding` metric + per-pass / per-cycle budget warnings.
- **Iteration auto-chain on gate-fail:** spurious gate failures no longer abort the autoloop — only REGRESSION + verdict signals halt.

v0.6.5 — Kit-bootstrap integrity gate (4-layer defense against un-installed-kit projects)

Closes the failure mode where a non-kit-aware agent (Copilot, plain-Claude) scaffolds a project manually, commits a `v0.N.N:` subject that satisfies reflex Gate 2, and proceeds end-to-end through prep-release + reflex — all while the project has zero kit scripts, no hooks, no skills, no CLAUDE.md, no `.portable-spec-kit/config.md` . Surfaced by an audit of the `psk-playground/urdu-stt-distillation` project which passed 3 reflex GRANTED verdicts despite being infrastructurally empty.

- **Layer 1 —** `install.sh` (unchanged, canonical installer).
- **Layer 2 —** `psk-release.sh` **Step 0:** `run_bootstrap_gate()` fires before `init_state` on `prepare` / `refresh` . FAIL exits 1 with full bootstrap report + remediation command — no release state written.
- **Layer 3 —** `reflex/lib/preconditions.sh` **Gate 0a:** runs after Gate 0 (self-test identity) and before Gate 1 (clean tree). Script resolution prefers target's own copy, falls back to kit's copy — catches the exact case where target has zero scripts.
- **Layer 4 — QA-Agent Dimension 16 "Kit-bootstrap integrity":** new mandatory dimension in `reflex/prompts/qa-agent.md` . Runs FIRST (before Phase 1 Plan). Halts pass with `QA-BOOTSTRAP-NN CRITICAL` finding on any C1-C7 fail.
- **New** `agent/scripts/psk-bootstrap-check.sh` (298 lines, shared helper). 7 checks (C1-C7): framework file + entry-point symlink, `.portable-spec-kit/config.md` , core 6 kit scripts executable, ≥10 cached skill files, pre-commit hook wired to psk-sync-check, all 9 `agent/*.md` pipeline files, test harness. Modes: default report, `--quiet` , `--json` , `--remediate` (auto-invokes parent install.sh). Kit-self auto-detected — skips symlink/config checks for the kit repo.

- **Register format improvements** in `reflex/lib/update-eval-trace.sh`: parses cycle from pass-name when `.cycle-meta` has stale `cycle=0`, adds `coverage.md` drill-down, extracts "Tests exercised:" summary line per pass (regression vectors + dimensions).
  Bypass for genuine emergencies: `PSK_BOOTSTRAP_CHECK_DISABLED=1`. Impact: structurally impossible now for an un-bootstrapped project to run prep-release or reflex.

v0.6.6 — Reflex Phase 0 pre-compute (claim-driven probing + reference-state diff + assumption surfacing)

Closes the meta-gap surfaced in v0.6.5 review: reflex's QA-Agent ran 3 GRANTED passes on an infrastructurally-empty project yet missed every kit-infrastructure gap because its probe set was a closed 16-dimension checklist from the AVACR paper. Adding dimensions reactively (Dim-15, Dim-16) fixes one case each but does not prevent the NEXT unknown class — QA always misses the bug outside its list. v0.6.6 moves reflex from "fixed checklist" to "probe-set derived from the project itself," **while keeping the 16-dim checklist as a safety net**. Three modes compose (each deterministic bash pre-compute — <1s, <100 tokens):

- **Mode A — Claim-driven probing (`reflex/lib/extract-claims.sh`):** walks every public doc (README, portable-spec-kit.md, SPECS, CHANGELOG, RELEASES, AGENT_CONTEXT, qa-agent.md) and extracts every claim with `probe_type` + `probe_target`. QA verifies each or files a finding. Unverified or vague claim = finding. Probe types: version-match, test-count, feature-count, capability-exists, file-count, install-works, dimension-count.
- **Mode B — Reference-state comparison (`reflex/lib/state-diff.sh` + `reflex/reference-state/speckit-project.yaml`):** static definition of what a complete speckit project must contain (required files + pipeline files + dirs + git hooks + entry-point symlinks + retired-file exclusions). State-diff compares actual vs reference; deltas pre-classified by severity; CRITICAL deltas auto-promote to findings. Kit-self auto-detected — skips user-project-only checks.
- **Mode C — Assumption surfacing (in `reflex/prompts/qa-agent.md`):** every pass writes `assumptions.md` listing implicit assumptions ("kit is installed," "test harness exits 0 on green," "every [x] feature actually implemented"). Each assumption gets a probe. Unverifiable assumption = MAJOR `QA-ASSUMPTION-NN` finding.
- **QA Phase 0 wiring:** both helpers run in `reflex/lib/spawn-qa.sh` BEFORE sandbox creation so `claims.yaml` + `state-diff.yaml` land in the pass directory pre-populated. QA reads these in Phase 0 — no LLM cost to re-derive. The 16-dim checklist runs as a safety net. CRITICAL state-diff short-circuits the 16-dim sweep of a structurally-broken project.

- **Dev-Agent expanded scope (in `reflex/prompts/dev-agent.md`):** Dev closes three gap classes per pass — (a) findings from `findings.yaml`, (b) unverified claims from `claims.yaml`, (c) state-diff deltas from `state-diff.yaml`. "Build, don't just patch" — restore missing infrastructure, build unfulfilled capabilities, add structure that makes assumptions self-verifying. Scope guardrails preserved (no out-of-SPECS features, protected files banned).
  Design discipline: extension within AVACR/Reflex, NOT a new framework. QA is still Critic, Dev is still Actor, adversarial-with-collaborative-outcome loop unchanged. What changed: QA's probe-set derivation (paper-defined → project-derived) and Dev's scope (patch-only → patch + build + remediate).

v0.6.7 — Reflex 7-layer Senior/Principal-level QA system

The user must NOT be QA of their own project. Reflex now operates with a 7-layer Senior/Principal-level QA system that derives all probes from the project's spec pipeline + kit toolkit + running system — zero human-curated lists. Builds on v0.6.6 Phase 0 pre-compute by adding four new verification layers and codifying Senior/Principal philosophies in the prompts.

- **Layer 1 — R→F→T pipeline integrity** (`reflex/lib/check-rft-integrity.sh`, deterministic bash, Phase 0). For every `[x]` feature in SPECS.md: maps to Rn in REQS.md (R→F traceability), has acceptance criteria block, has `agent/design/f{N}-*.md` design plan, Tests column references real test file, test contains non-trivial assertions (no `toBeDefined` trivia / `test.skip` / TODO markers), test exits 0, has `[x]` mark in TASKS.md. Each break = CRITICAL finding. Output: `rft-integrity.yaml`.
- **Layer 2 — Bidirectional doc ↔ code consistency** (`reflex/lib/doc-code-diff.sh`, deterministic bash, Phase 0). Doc-to-code: every named file/script/version/count/capability claim → grep code for backing. Code-to-doc: every kit script + every `[x]` feature → at least one doc surface mentions it. Numerical-claim cross-check (test count consistency). Drift = finding. Output: `doc-code-diff.yaml`.
- **Layer 3 — Behavioral per-feature verification** (qa-agent.md Phase 2 mandate, LLM). Per `[x]` feature: 1 happy + 5 edge cases + 3 adversarial inputs + 1 integration scenario. **Independence rule**: design own test BEFORE reading Dev's. Output: `behavioral-tests/F{N}-*`.
- **Layer 4 — Test-quality audit** (qa-agent.md Phase 3 mandate, LLM). After Layer 3, audit Dev's test: would it catch QA's adversarial inputs? Are mocks honest? Coverage adequate? Inadequate Dev test = MAJOR `QA-TEST-INADEQUATE-F{N}` even if currently passing. Output: `test-quality-audit.yaml`.
- **Layer 5 — Cross-feature integration probes** (qa-agent.md Phase 2 mandate). Identifies pairs of `[x]` features that interact (auth × data, search × permissions).

Surfaces compose-time bugs single-feature tests miss. Output: `integration-probes.md`.

- **Layer 6 — Production readiness via kit tools** (`reflex/lib/spawn-qa.sh` Phase 0 capture). Invokes `psk-sync-check.sh --full`, `psk-doc-sync.sh`, `tests/test-release-check.sh agent/SPECS.md`, `psk-code-review.sh`. Every PSK error code, MISSING in doc-sync, R→F→T break in release-check becomes evidence. Output: `kit-tools-output/{sync-check,doc-sync,release-check,code-review}.txt`.
- **Layer 7 — External reality check** (qa-agent.md Phase 2 mandate, LLM + WebSearch/WebFetch). For code touching network/dependencies/standards: cross-check OWASP Top 10, CVE feeds, framework deprecations (e.g., transformers `as_target_processor()` removed in v4.44), best-practice patterns. Output: `external-research.md`.
- **Senior/Principal-level philosophies** codified in qa-agent.md ("the user MUST NOT be QA of their own project") and dev-agent.md ("Senior/Principal engineer who closes gaps QA surfaces and hardens implementation against future passes"). 7 operating principles each, 4 forbidden moves, three gap classes per pass for Dev (fix findings + build unfulfilled claims + remediate state-diff deltas).
- **Architecture plan committed:** `agent/design/f70-reflex-senior-engineer-qa.md` documents context, philosophies, all 7 layers, what stays from v0.6.5/v0.6.6, what was discarded, v0.6.7-v0.6.13 sequencing. Companion Dev plan at `agent/design/f70-reflex-senior-engineer-dev.md`. Layer 4 hardening log shipped (`reflex/history/hardening-log.md`).
  Short-circuit rule: if Layer 1 (rft-integrity) or Mode B (state-diff) shows CRITICAL deltas indicating structural break, halt the pass with those findings and skip the 16-dim sweep — fix structure first, behaviors second.

v0.6.8 — Reflex 7-layer full implementation: Layer 3/5/7 QA seed-helpers + Layer 7 Dev ADL extraction

Completes v0.6.7's architecture by shipping deterministic helpers for Layers 3, 5, 7 (QA) and Layer 7 (Dev). All 7 layers now have deterministic-bash seed-helpers wherever possible — LLM creative work goes to genuine reasoning, not derivation.

- `reflex/lib/scaffold-behavioral-tests.sh` **(QA Layer 3):** for every `[x]` feature in `agent/SPECS.md`, generates `behavioral-tests/F{N}-test-plan.md` skeleton with the 1+5+3+1 structure (1 happy + 5 edge + 3 adversarial + 1 integration). Pulls acceptance criteria from `### F{N}` blocks; flags missing criteria as Layer 1 finding. QA fills TODOs with concrete probes — independence rule: design BEFORE reading Dev's test. Skeleton saves ~5K tokens/pass.
- `reflex/lib/identify-integration-probes.sh` **(QA Layer 5):** scans SPECS for feature interactions via design-plan + criteria-block cross-references. Emits `integration-`

`probes.md` with candidate pairs. QA validates pairs, drops spurious, designs scenarios. Includes "common integration patterns" reference.
- `reflex/lib/external-research.sh` **(QA Layer 7):** scans project manifests (`package.json` , `requirements.txt` , `pyproject.toml` , `go.mod` , `Cargo.toml` , `Dockerfile`) and emits `external-research.md` with seeded WebSearch/WebFetch queries: per-dependency CVE searches, framework-changelog URLs, OWASP Top 10 mapping for project class. Stack-specific recent-vulnerability-class hints (e.g., prompt injection per OWASP LLM Top 10).
- `reflex/lib/auto-extract-adl.sh` **(Dev Layer 7):** post-pass, scans Dev's commits for substantive bodies (≥200 chars) containing rationale keywords. Drafts ADL entries to `agent/.release-state/adl-drafts.md` for kit-maintainer review.
- **Phase 0 produces 8 artifacts:** `spawn-qa.sh` invokes all helpers BEFORE sandbox creation. Pass dir contains `claims.yaml` , `state-diff.yaml` , `rft-integrity.yaml` , `doc-code-diff.yaml` , `behavioral-tests/` , `integration-probes.md` , `external-research.md` , `kit-tools-output/` when QA spawns.
- **Field-tested on kit-self:** Layer 3 produces 70 test-plan skeletons in <1s · Layer 5 finds 77 candidate pairs · Layer 7 QA gracefully detects stack signals · Layer 7 Dev drafts ADL entries when commits have rationale.

v0.6.9 — The kit is audited and improved by the reflex it built

The architecture built across v0.6.5-v0.6.8 — Phase 0 deterministic helpers + Senior/Principal-level QA philosophy + paired Dev operating system — was designed to find drift in user projects. v0.6.9 turned that exact machinery on the kit itself. The same QA-Agent that catches bugs in your code caught real bugs in the code that builds the QA-Agent.

- **Phase 0 helpers ran on kit-self** and exposed **180 mechanical drift items** the kit's own existing mechanical gates had passed clean for months: 170 R→F→T pipeline breaks (55 features missing `### F{N}` blocks, 45 missing design plans) + 10 doc↔code drifts including 1 LIVE rename incompleteness from v0.6.2 entry consolidation.
- **QA-Agent was spawned** against the kit and filed **5 findings** (3 MAJOR + 2 MINOR), prioritized for the field-test scope.
- **Dev-Agent closed 2 inline:** `QA-DOC-FT-02` live `kit-loop-state.yml` → `loop-state.yml` rename (4 files); `QA-TEST-INADEQUATE-F1` Layer-4 hardening shipping new `PSK018` per-feature pairwise check (advisory by default; `PSK_FEATURE_CRITERIA_STRICT=1` promotes to hard fail). The PSK018 hardening would have caught a "69 of 70 acceptance-criteria blocks deleted" silent ship under the existing shallow gate.
- **Dev deferred 3 to v0.6.13 ARB** in `agent/TASKS.md` : `QA-PIPE-FT-01-ARB` (backfill 55 missing criteria blocks — kit's own RFT debt) + `QA-INT-F70-F52-ARB` + `QA-INT-`

`F70-F1-ARB` (behavioral integration probes need full QA+Dev pass, not field-test scope).

- **Layer 5 hub-suppression added** in `reflex/lib/identify-integration-probes.sh`: features appearing in ≥10 candidate pairs are hubs (e.g. F64 design pipeline cross-refs everything; F70 reflex links many) — pairs involving hubs are mostly noise, suppressed into a separate callout. Result on kit-self: 77 → 13 pairs.
- **Field-test artifacts committed** in `reflex/history/field-test-v0.6.8/` as permanent evidence: claims, state-diff, rft-integrity, doc-code-diff, behavioral-tests/, integration-probes, external-research, kit-tools-output, findings, signoff, dev-trace.
- **Hardening event H-001 logged** in `reflex/history/hardening-log.md`.
  A tool sufficient to audit user projects must be sufficient to audit the project that built it — v0.6.9 proved that proposition. v0.6.13 then closed the kit's own RFT debt + flipped PSK018 strict, locking in the standard the architecture proved.

v0.6.13 — Nested per-cycle history layout + Dim 23 + cycle-summary aggregator + autoloop convergence

Closes the meta-gap class user surfaced in cycle-01 trace audit (5 issues + 4 sub-gaps), formalizes Dim 23 so QA self-evolves to catch the class going forward, migrates reflex history to a nested per-cycle directory layout, and drops the hard-3-iter autoloop cap in favor of convergence-based stopping.

- **Dim 23 — Auditor-output hygiene (MANDATORY):** five new probes (register hygiene · cost data persistence · parallel-directory disambiguation · cross-surface terminology consistency · self-output validation loop) catch the meta-gap class previous 22 dimensions didn't probe. `reflex/prompts/qa-agent.md` v0.6.13 grows from 22 → 23 dimensions; QA's own self-evolution loop (Dim 18) is now the mechanism by which new dimensions are added.
- **Nested per-cycle history layout:** `reflex/history/cycle-NN-pass-NNN/` → `cycle-NN/pass-NNN/` (autoloop) and `standalone/pass-NNN/` (single-pass). Sandbox mirrors history layout. Per-cycle parent dir hosts a `summary.md` aggregator (`reflex/lib/cycle-summary.sh`). Flat identity (`cycle-NN-pass-NNN`) reserved for git branches, CSV pass id, and `[source: ...]` commit trailers — decouples branch/identity from on-disk layout.
- **Autoloop runs until convergence:** hard 3-iter cap dropped. Primary stops are convergence-based (GRANTED · REGRESSION · findings_floor · plateau · fix-rate drop). `convergence.max_iterations_safety` default 20 is escape hatch only.
- **Cycle-01 trace audit fixes (9/9):** pass-002 visible in register as INCOMPLETE (was silently dropped); "Autoloop cycle" → "Reflex cycle" terminology unified; sandbox-vs-history disambiguation in `17-reflex.md`; `score.sh` `grep -c || echo 0` multi-line bug fixed (`head -1`); `doc-code-diff.sh` path-with-spaces fix (10× scan coverage 12 → 120 docs); `psk-doc-sync.sh --strict` tightened (Missing>0 only).

- **First successful Dev-Agent run end-to-end:** `cycle-01-pass-004` closed `QA-NOISE-RE-01` bucket A (NOISE_RE backtick-prefix SHA gap), 1 commit, all 6 mechanical gates green, fast-forward auto-merge to main — first time QA→Dev→merge loop closed without manual intervention.
- **3-layer QA→Dev contract enforcement:** `resume-qa` requires `findings.yaml` + `signoff.md`; `resume-dev` requires `dev-trace.md`; `loop.sh` Phase 4 INCOMPLETE detection. Closes "Dev never spawned on partial QA timeout" silent-skip pattern observed in `cycle-01-pass-002`.
- **Tests:** 1633 passing (1488 framework + 145 benchmarking). Sync-check 19/19. Release-check 70/70 features pass with R→F→T coverage.
- **Tagline:** *Auditor-output hygiene formalized as Dim 23; reflex history reshaped per-cycle; autoloop runs to convergence.*

v0.6.12 — Reflex iter-1 convergence anchor + close all 15 v0.6.11 audit findings

Anchor version for the reflex iter-1 convergence cycle. Consolidates 8 refresh-release commits from v0.6.11 baseline. All 15 audit findings (5 pre-existing ARBs + 10 iter-1 QA findings) resolved or properly deferred; kit's own QA infrastructure self-validated end-to-end via reflex iter-1 self-test on kit-self.

- **All 10 iter-1 QA findings closed:** QA-REL-NONDETERM-01 (CRITICAL — self-cleaning rft-cache), QA-KIT-RFT-01 (anchored regex, 70 false-positives → 0), QA-KIT-DCD-02 (DOC_SURFACES expanded 6→11), QA-PERF-PHASE0-01 (persistent test-ref cache, 10.9× speedup), plus 5 MINOR + Option C thematic test-suite split (closes QA-TEST-COUPLING-01).
- **5 pre-existing ARBs closed:** QA-ARCH-01 (Chrome → WeasyPrint per ADR-006), G2/G5 (verified already-shipped), QA-INT-F70-F1-ARB (C1.5 symlink target validation), QA-INT-F70-F52-ARB (layered R→F→T documented as intentional).
- **Cost/perf wins:** persistent test-ref cache cold 54s → warm 5s = 10.9× speedup. Phase 0 wall-clock 74s → ~25s on subsequent invocations. Dim 17 cost/perf audit + Dim 18 philosophy self-audit added to QA prompt + Dev mirror principles 8/9. pass_score (Dim 17 ranking) v3 summary.csv schema.
- **Tests:** 1621 passing.
- **Tagline:** *Iter-1 self-audit complete — kit closed every finding it surfaced about itself.*

v0.6.11 — Reset allowlist + iter-aware refresh-release optimization

Two kit-infrastructure improvements driven by usage friction noticed during v0.6.10 wrap-up. Both reduce maintenance burden + cost without changing reflex semantics.

- **Reset reflex — allowlist-based collection:** `reflex/lib/reset.sh` switched from hardcoded glob lists to allowlist-based `find -mindepth 1 -maxdepth 1` traversal. New

`HISTORY_KEEP=("hardening-log.md")` allowlist preserves kit's structural-defense audit memory across resets. Symmetric `--reset-hardening` flag for true clean-slate.
- **Iter-aware release-prep:** `reflex/lib/loop.sh` Phase 1 branches on `ITER`: iter 1 = `prepare` (version bump), iter 2+ = `refresh` (no bump). Single converged reflex cycle no longer inflates versions.
- **Doc-coverage backfill:** new "v0.6 Capability Reference" section in `docs/work-flows/17-reflex.md`. Closes 24 cumulative `psk-doc-sync.sh` coverage gaps from v0.6.5-v0.6.10.

v0.6.10 — Closing v0.6.9 ARBs: kit's own RFT debt backfilled + PSK018 strict by default

v0.6.9's field-test surfaced that the kit itself had accumulated RFT (Requirements → Features → Tests) debt: 55 of 70 features lacked their per-feature `### F{N}` acceptance criteria block in `agent/SPECS.md`. The new Layer-4 hardening shipped in v0.6.9 (`PSK018`) caught it as advisory; v0.6.10 acted on it.

- **Backfilled all 55 missing `### F{N}` acceptance criteria blocks** in `agent/SPECS.md` (F1–F55, 3-5 testable bullets each, ~200 lines), closing `QA-PIPE-FT-01-ARB`. F56–F70 blocks already existed and were preserved unchanged.
- **Flipped `PSK018` advisory → strict-by-default** in `agent/scripts/psk-sync-check.sh`. Default: `local strict="${PSK_FEATURE_CRITERIA_STRICT:-1}"`. Future commits adding an `[x]` feature without matching `### F{N}` block now fail pre-commit. Opt-out via `PSK_FEATURE_CRITERIA_STRICT=0` for genuine emergencies.
- **Layer 1 R→F→T integrity on kit-self:** 55 C2-criteria breaks → 0. Total drift items 170 → 110 (remaining are C1 R→F + C3 design-plan + minor edges, addressable in future versions).
- **Tests:** 1604 passing (1459 framework + 145 benchmarking). Sync-check 19/19. Release-check 70/70 features pass with R→F→T coverage.
- **Still deferred (v0.6.13+ ARB):** `QA-INT-F70-F52-ARB` (reflex × test-harness behavioral integration probe) + `QA-INT-F70-F1-ARB` (reflex × framework-file behavioral integration probe). Both require full QA + Dev pass scope — not field-test scope. Tracked in `agent/TASKS.md`.

What the Framework File Contains

The `portable-spec-kit.md` file is the single source of truth for your development methodology. It contains:

Section	What It Governs

User Profile	Personalized AI — GitHub auto-detect, communication style, working pattern, AI delegation
Git & GitHub Rules	When to commit, push, ask before critical ops
Security Rules	Secret handling, .env practices, code security
Versioning	Two-level: framework v0.N.x (each publish) + release v0.N (milestones), auto-restructure on pull
Task Tracking	Add tasks FIRST, then work. Module-based organization.
Testing Standards	Coverage targets, edge case checklist, mock rules, self-validation
Code Quality	Review checklist, naming conventions, deployment checklist
Error Handling	Structured errors, logging, error boundaries, user-friendly messages
Branch & PR Workflow	Feature branches, PR format, squash merge, CI-before-merge requirement
CI & Community Contributions	GitHub Actions ci.yml template, stack-aware test commands, branch protection, PR workflow, contribution validation
Spec-Based Test Generation	Forward-flow stub generation, per-feature acceptance criteria format, stack-aware stubs, stub completion gate
Dependencies	Bundle size checks, lock files, audit, avoid unnecessary deps
Project Templates	6 agent files + README + 8 source code structures (Web, Python, Mobile, Android, iOS, Full Stack, Full Stack + Mobile, Document)
Auto-Scan	Detects existing projects, creates missing files, restructures to standard — 16 work flow docs
Agent Behavior	Guide don't enforce, silent tracking, retroactive spec-filling, R→F→T traceability

Multi-Agent Support

One source file — `portable-spec-kit.md` — works with every AI coding agent via symlinks:

Agent	File Created	How

Claude Code	`CLAUDE.md`	Symlink → portable-spec-kit.md
GitHub Copilot	`.github/copilot-instructions.md`	Symlink → portable-spec-kit.md
Cursor	`.cursorrules`	Symlink → portable-spec-kit.md
Windsurf	`.windsurfrules`	Symlink → portable-spec-kit.md
Cline	`.clinerules`	Symlink → portable-spec-kit.md

macOS/Linux: Uses symlinks — edit `portable-spec-kit.md`, all agents read the update instantly.

Windows: Uses copies — re-run install command after editing, or let the AI agent keep files in sync.

Complete Flow

1. **Install** — one command downloads `portable-spec-kit.md` + creates symlinks for all agents (Claude, Copilot, Cursor, Windsurf, Cline)

↓

2. **Open any AI agent — auto-scaffolds** — detects environment, creates WORKSPACE_CONTEXT.md, creates `agent/` with 6 files (AGENT, AGENT_CONTEXT, SPECS, PLANS, TASKS, RELEASES), project dirs (src/, tests/, docs/), README, .gitignore, .env.example, tests/test-release-check.sh

↓

3. **Work however you want** — "build me X" → agent adds to TASKS.md, builds, tests. "What should I do next?" → agent walks through spec-persistent flow. Never blocks.

↓

4. **Session ends** — agent updates AGENT_CONTEXT.md with progress, decisions, what's next

↓

5. **Come back later** — days, weeks, months. Agent reads AGENT_CONTEXT.md: "Here's where we left off." Continues exactly where you stopped.

↓

6. **Edit the framework** — update portable-spec-kit.md, all 5 agent symlinks read the change instantly. Standards evolve with you.

Project Structure

```
your-project/
├── portable-spec-kit.md    ← Framework file (source)
├── .portable-spec-kit/     ← Kit config (user profiles, per-user)
│   └── user-profile/
│       └── user-profile-{username}.md
├── CLAUDE.md               ← Symlink (Claude Code)
├── .cursorrules            ← Symlink (Cursor)
├── WORKSPACE_CONTEXT.md    ← Auto-created (workspace state)
├── README.md               ← Auto-created (standard structure)
│
├── agent/                 ← Auto-created (project management)
│   ├── AGENT.md           ← Project rules, stack, brand
```

```
│   ├── AGENT_CONTEXT.md   ← Living state (updated every session)
│   ├── SPECS.md           ← Requirements & features
│   ├── PLANS.md         ← Architecture, methodology & research
│   ├── TASKS.md           ← Task tracking (module-based)
│   └── RELEASES.md         ← Version history
│
├── src/                   ← Your code
├── tests/                 ← Your tests
├── ard/                   ← Architecture docs (HTML + PDF)
└── ...
```

The 6 Agent Files

File	Purpose	When
`AGENT.md`	Project-specific instructions — stack, rules, brand, AI config	Setup
`AGENT_CONTEXT.md`	Living state — done, next, decisions, blockers, file structure	Every session
`SPECS.md`	What to build — requirements, features, acceptance criteria, scope	Before dev
`PLANS.md`	How to build — architecture, data model, phases, methodology, research, decision log	Before dev
`TASKS.md`	Module-based task tracking with progress summary table	During dev
`RELEASES.md`	Version changelog (categorized), test results, deployment log	End of version

How to Work in Spec-Persistent Development

Phase 1: Specification

> **Goal:** Define WHAT you're building before touching code.

Start every project by writing `agent/SPECS.md`. This is a conversation between you and the AI:

You Say	AI Does

"I want to build a resume editor"	Writes requirements, features table, scope boundaries, acceptance criteria in SPECS.md
"Add AI-powered bullet rewriting"	Adds feature to specs, estimates priority, updates scope
"That's out of scope for v1"	Moves to "Out of scope (future)" section

Rule: No code is written until SPECS.md has requirements and acceptance criteria.

Phase 2: Planning

> **Goal:** Define HOW you're building it — architecture, data model, phases.

AI writes `agent/PLANS.md` based on approved specs:

Section	What AI Fills In
Stack	Recommends tech (frontend, backend, DB, hosting) with trade-offs. You approve.
Architecture	System design, component relationships, data flow
Directory Structure	Exact file/folder layout for the chosen stack
Data Model	Tables, types, relationships
API Endpoints	Method, path, description for each route
Build Phases	Phased delivery plan — foundation first, features later
Security	Auth, CORS, input validation, secret handling

Rule: Deployment is deferred to release time. Don't discuss hosting during planning.

Phase 3: Execution

> **Goal:** Build, test, and track every task.

AI creates `agent/TASKS.md` with module-based breakdown:

```
## Module 1: Project Setup
| # | Task | Status |
```

```
|---|------|:------:|
| 1.1 | Initialize Next.js + Tailwind | [x] |
| 1.2 | Create component structure | [ ] |

## Module 2: Core Features
| # | Task | Status |
|---|------|:------:|
| 2.1 | Build editor canvas | [ ] |
```

Execution rules enforced by the framework:

Rule	What It Means
Tasks first	When you ask for something, AI adds it to TASKS.md BEFORE starting work
Self-validate	AI runs tests, fixes failures, only presents stable results
Test everything	Every feature gets tests — unit, integration, UI interaction
Edge cases	Empty inputs, boundaries, XSS, unicode, error paths — all tested
Coverage targets	Backend 98%+, frontend logic 98%+, UI components 85%+
No broken features	User should NEVER discover something doesn't work
Context updated	AGENT_CONTEXT.md updated after significant work, after commits, and before any push
Flows updated	docs/work-flows/ updated if any work flow changed — flows always match code

Phase 4: Tracking

> **Goal:** Log what was delivered, tested, and deployed.

At end of each version, AI updates `agent/RELEASES.md`:

```
## v0.2 — AI Features (March 30, 2026)

### Summary
Added JD-tailored resume, PDF generation, 673 tests.

### Changes
- **Frontend:** JD Tailor modal, version manager, skills tags
```

```
- **Backend:** 3 PDF engines, expand API, fetch-url API
- **AI:** Bullet format detection, deduplication, length matching

### Tests
- 673 tests passing, 95% coverage

### Deployment
- Deployed to: aqibmumtaz.github.io
- Date: 2026-03-30
```

Daily Workflow Summary

1. **Start session** — AI reads agent/AGENT.md + AGENT_CONTEXT.md. Knows exactly where you left off.

↓

2. **You give instructions** — "add dark mode", "fix scoring bug", "build PDF export"

↓

3. **AI adds to TASKS.md** first, then builds, tests, and shows results with coverage

↓

4. **You review** — approve, adjust, or ask for changes. AI fixes and re-tests.

↓

5. **You say "commit"** — AI commits with descriptive message. "push" sends to remote.

↓

6. **Session ends** — AI updates AGENT_CONTEXT.md. Next session picks up seamlessly.

> **The result:** Every project follows the same spec → plan → build → test → track pipeline. Nothing is ad-hoc. Nothing falls through cracks. Context never lost.

Agent Behavior — Guide, Don't Enforce

The framework is **not rigid**. The AI agent adapts to how you want to work:

> **Core principle:** The user can start anywhere — jump into coding, give direct tasks, ask questions, or follow the full spec-persistent flow. All valid. The agent adapts to the user, not the other way around.

You Do	Agent Does
"Build me a login page"	Adds to TASKS.md → builds it → tests it → shows results
"Fix this bug"	Adds to TASKS.md → fixes → marks done
"What should I do next?"	Walks through spec-persistent process: define specs → plan → tasks
"What's the status?"	Shows from TASKS.md + AGENT_CONTEXT.md
Come back after weeks	Reads AGENT_CONTEXT.md → summarizes where you left off
Never wrote specs	Retroactively fills SPECS.md from what's built — no complaints

Silent Tracking

Even if you never mention the process, the agent:

- Adds every task to TASKS.md before starting
- Marks tasks done when complete
- Updates AGENT_CONTEXT.md after completing work, committing, and before pushing
- Fills gaps in SPECS.md and PLANS.md retroactively
- Surfaces process naturally: "I've added this to TASKS.md", "Based on SPECS.md, we still have..."

> **User's time is sacred.** Agent does 90% of the work — writes specs, plans, tasks, tests, docs. User reviews 10%. Never ask the user to write specs or plans — the agent drafts everything, user approves or adjusts.

File Creation/Update Rule

This single rule governs how the agent handles ALL managed files (`WORKSPACE_CONTEXT.md`, `README.md`, and all `agent/` files):

Scenario	Action
File does not exist	Create from standard template, fill in known details
File exists but wrong structure	Restructure to match template — **retain all existing content**, never lose data
File already matches template	Leave as-is

> **Never lose content.** When restructuring an existing file, every detail, decision, and note is preserved — just reorganized into the standard sections.

Customization

User Profile — Personalized AI Experience

On first session, the agent fetches your GitHub identity and asks 3 quick questions. Press Enter to use recommended, or type your own:

Question	Options
Communication style?	(a) direct and concise ← RECOMMENDED (b) direct, data-driven, comprehensive with tables (c) conversational and collaborative Press Enter to use recommended (a)
Working pattern?	(a) iterative — start small, expand ambitiously ← RECOMMENDED (b) plan-first — full specs before code (c) prototype-fast — build quick, refine later Press Enter to use recommended (a)
AI delegation?	(a) AI does 70%, user guides 30% ← RECOMMENDED (b) AI does 90%, user reviews 10% (c) 50/50 collaboration Press Enter to use recommended (a)

Saved to:

```
~/.portable-spec-kit/user-profile/user-profile-janesmith.md         (global — asked once)
workspace/.portable-spec-kit/user-profile/user-profile-janesmith.md (committed — persists)
```

> **Personalized from first session.** Profile asked once, saved globally. On new projects: shown and confirmed — keep or customize per project. Customized profiles save to workspace. Press Enter through everything for recommended defaults.

Your Projects

Project-specific details go in `agent/AGENT.md` (auto-created per project). This keeps the framework file portable while each project has its own stack, brand, and rules:

```
## Stack
| Layer | Technology |
|-------|-----------|
| Frontend | Next.js + TypeScript |
| Backend  | FastAPI |

## Brand
- Primary: #1B2A4A
- Accent: #4a6fa5
```

Dev Stack Adaptation

The framework uses stack-agnostic language for testing, building, and linting. Each project's `agent/AGENT.md` defines the actual commands:

Framework Says	Project Defines
"All tests passing"	`npm test` / `pytest` / `go test`
"Type checking: zero errors"	`npx tsc --noEmit` / `mypy`
"Linting: zero errors"	`eslint` / `ruff` / `golint`
"Build succeeds"	`npm run build` / `docker build`
"PDF generation tool"	WeasyPrint / Puppeteer / wkhtmltopdf

> **Stack-agnostic by design.** The framework never assumes a specific language, build tool, or test runner. It defines *what* to check — each project defines *how*.

Sharing & Syncing the Framework

Recommended: Dedicated GitHub Repo

📦 Single-Source-of-Truth Repo

One repo holds portable-spec-kit.md. All projects pull from it. Updates pushed back from any project.

```
# Setup (once)
mkdir portable-spec-kit && cd portable-spec-kit
git init && cp /path/to/portable-spec-kit.md .
git add . && git commit -m "Framework v1.0"
gh repo create portable-spec-kit --private --push

# Pull into any project
curl -sO https://raw.githubusercontent.com/you/portable-spec-kit/main/portable-spec-kit.md

# Push updates back from any project
cp portable-spec-kit.md ~/portable-spec-kit/
cd ~/portable-spec-kit && git add . && git commit -m "Updated from ProjectX" && git push
```

✓ Version controlled ✓ Pull anywhere ✓ Change history ✓ Cross-machine sync

Other Options

Method	How	Best For
Git Submodule	`git submodule add repo .framework`	Auto-sync across repos
GitHub Gist	`gh gist create portable-spec-kit.md`	Quick sharing, browser editable
Shell Alias	`alias init-dev='curl -sO URL/portable-spec-kit.md'`	One-command setup

Update Workflow Across Projects

1. **Discover new practice** while working on Project A (e.g., new testing rule)

↓

2. **AI updates portable-spec-kit.md** in Project A with the new rule

↓

3. **Push to shared repo** — single source of truth updated

↓

4. **Pull in Project B** — instantly has the new rule. AI follows it immediately.

Portable vs Project-Specific

In portable-spec-kit.md (Portable Framework)	In agent/AGENT.md (Per Project)
Git/push/commit rules	Tech stack choices
Security practices	Brand colors & fonts
Testing methodology	AI model/provider config
Naming conventions	Dev server port
Project templates & auto-scan	Deployment specifics
Code review standards	Project-specific rules
Spec-driven development flow	API endpoints & data model
Version numbering scheme	Database schema

Examples Included

The repo includes two example projects showing the framework in action:

Example	What It Shows	Best For
`examples/starter/`	Fresh project right after setup. Self-documenting README explains every file and directory.	Understanding the framework — **start here**
`examples/my-app/`	Mid-development Next.js + Supabase project. 11/16 tasks done, 24 tests at 92% coverage.	Seeing a realistic project in progress

Starter Project Structure

```
examples/starter/
├── portable-spec-kit.md    ← Framework file (source)
├── .portable-spec-kit/     ← Kit config (user profiles)
│   └── user-profile/
├── CLAUDE.md               ← Symlink (Claude Code)
├── .cursorrules            ← Symlink (Cursor)
├── WORKSPACE_CONTEXT.md    ← Auto-created workspace state
├── README.md               ← Self-documenting (explains everything)
├── agent/
│   ├── AGENT.md           ← Stack: TBD (waiting for your specs)
│   ├── AGENT_CONTEXT.md   ← Status: "Setup — waiting for specs"
│   ├── SPECS.md           ← Empty — ready for requirements
│   ├── PLANS.md        ← Empty — ready for architecture & methodology
│   ├── TASKS.md           ← 1/5 tasks (project initialized)
│   └── RELEASES.md         ← v0.1 placeholder
```

Mid-Development Project

```
examples/my-app/
├── agent/
│   ├── AGENT.md           ← Next.js + Supabase configured
│   ├── AGENT_CONTEXT.md   ← v0.1: 11/16 tasks, 24 tests
│   ├── SPECS.md           ← 8 features, priorities, criteria
│   ├── PLANS.md        ← Data model, APIs, 3 phases
│   ├── TASKS.md           ← 5 modules with progress
│   └── RELEASES.md         ← Categorized changelog
```

One file. Any project. Your standards.
The framework travels with you — consistent engineering practices across every project, every team, every AI session.

Portable Spec Kit

Thank You

Author: Dr. Aqib Mumtaz, Ph.D.

Specialization: Ph.D. Computer Science — Artificial Intelligence

Research: Multimodal AI, Healthcare AI, Autonomous Surveillance

Email: aqib.mumtaz@gmail.com

LinkedIn: linkedin.com/in/aqibmumtaz

GitHub: github.com/aqibmumtaz

v0.6.38 • May 2026